

Code Review Report

Language: python | Review mode: openai | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 14:14 UTC

Overall Assessment

The code review has identified 6 security findings, with 4 Critical, 1 High, and 1 Medium severity issues. These findings pose significant security risks, including hardcoded sensitive information, SQL injection vulnerabilities, and potential code injection attacks.

Merge readiness: Not Ready

Code health: 100/100

Findings Summary

Total findings: 6

Risk score: 0

Severity breakdown: Low 0, Medium 0, High 0, Critical 0

Type	Severity	Line	Message	Recommendation
Security	Critical	1	Hardcoded sensitive information (password)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	2	Hardcoded sensitive information (api_key)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	5	SQL injection vulnerability due to string concatenation	Use parameterized queries or prepared statements to prevent SQL injection.
Security	Critical	6	Use of eval() function with user input	Avoid using eval() with user input, as it can lead to code injection attacks. Use safer alternatives like ast.literal_eval() or json.loads() instead.
Security	Medium	7	Use of exec() function	Avoid using exec() whenever possible, as it can pose a security risk if used with untrusted input. Consider using safer alternatives or refactoring the code to avoid dynamic execution.
Security	High	10	Use of subprocess with shell=True	Avoid using shell=True with subprocess, as it can lead to shell injection attacks. Use the subprocess.run() function with the 'args' parameter instead, passing the command and arguments as a list.

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
Security	Critical	Hardcoded sensitive information (password) poses a significant security risk	Store sensitive information securely using environment variables or a secrets manager
Security	Critical	Hardcoded sensitive information (api_key) poses a significant security risk	Store sensitive information securely using environment variables or a secrets manager
Security	Critical	SQL injection vulnerability due to string concatenation	Use parameterized queries or prepared statements to prevent SQL injection
Security	Critical	Use of eval() function with user input can lead to code injection attacks	Avoid using eval() with user input, use safer alternatives like ast.literal_eval() or json.loads() instead
Security	High	Use of subprocess with shell=True can lead to shell injection attacks	Use the subprocess.run() function with the 'args' parameter instead, passing the command and arguments as a list
Security	Medium	Use of exec() function can pose a security risk if used with untrusted input	Replace exec() with a safer alternative, such as a function call or a dictionary-based approach

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```